

ping-v2 — 프로젝트 완주 기록

분석할 수 없는 데이터에서 대시보드까지 · 2026-07-29 ~ 2026-08-25

목표 — 데이터 파이프라인과 분석. 웹앱은 실데이터를 만드는 수단이었다

결과 — 웹앱 배포 · 실유저 데이터 적재 · 합성 1억 2,370만 행 · raw → stg → mart → 루커 대시보드 5쪽까지 완주

기간 — 28일 (작업일 15일) · 커밋 143개

이 문서가 아닌 것 — 입문 설명은 PROJECT-REPORT.pdf (12장), 합성 데이터 만드는 법은 synthetic-data-build-spec.pdf (17장), 검수는 EDA-final-12m-v5.pdf

00 · 한 장 요약

27 / 28

완료 단계
(웹앱 W0~19 · 파이프라인
P0~7)

1억 2,373만

BigQuery raw 행
8.74 GiB

5쪽

루커 대시보드
mart 8표 연결

89

결정 노드
(뒤집힌 것 4)

구 서비스 데이터로는 답할 수 없는 질문이 있었다. 투표한 사람과 결제한 사람을 같은 user_id 로 이을 수 없었고 (결제자 59,192명 중 99.3%가 애초에 투표가 불가능한 학교 소속), 탈퇴 기록 70,764건에는 유저 식별자가 아예 없었다. 데이터를 더 얻어도 풀리지 않는 구조 문제였다.

그래서 순서를 이렇게 잡았다 — 결함을 담은 스키마를 설계하고 → 그 위에 웹앱을 올려 실데이터를 만들고 → 규모를 시험할 합성 데이터를 엮고 → 파이프라인으로 흘려 → 대시보드까지 세운다.

28일 만에 그 전부가 섰다. 파이프라인은 raw → stg → mart 세 층이 Airflow DAG 3개로 돌고, 루커 스튜디오 5쪽이 mart만 바라본다.

이 문서에서 제일 볼 만한 장은 12장이다

이 프로젝트에서 실제로 배운 것은 성공한 설계가 아니라 조용히 틀린 것들이다 — 오류가 안 나고, 파이프라인은 초록불이고, 숫자만 틀린 경우. 12장에 전부 모았다.

00 한 장 요약

01 무엇을 하려던 프로젝트였나 — 네 가지 의도

02 출발점 — 구 서비스에서 막힌 것

03 전체 연표 — 28일, 다섯 국면

04 완주표 — 웹앱 20단계 · 파이프라인 8단계

05 산출물 ① 스키마 — 40표

06 산출물 ② 웹앱 — 개인정보를 받지 않는 MVP

07 산출물 ③ 합성 데이터 — 아홉 판본의 기록

08 산출물 ④ 파이프라인 — 세 층과 DAG 셋

대시

- 09 산출물 ⑤ 보드 — 루커 5쪽 신규 완료
- 10 산출물 ⑥ 문서 체계 — 89개 결정 노드
- 11 검증 체계 — 층층이 쌓은 검사 다섯
실패
- 12 기록 — 조용히 틀린 것들 핵심
- 13 숫자로 본 프로젝트
- 14 관통하는 교훈 다섯
- 15 남은 일 — 정직하게
- 16 이 프로젝트가 증명한 것

01 · 무엇을 하려던 프로젝트였나

네 가지 의도가 있었고, 그 순서가 프로젝트 구조를 결정했다.

의도	내용	결과
① 파이프라인을 직접 만들어본다	분석은 해왔지만 엔지니어링은 경험이 없었다 . BigQuery · Airflow 같은 도구를 실제로 써서 웹앱 → DW → 대시보드 전 구간을 세워 본다	달성 P0~P7 완주
② 데이터를 직접 설계한다	외부 데이터는 구조가 이미 정해져 있어 필요한 연결고리를 만들 수 없다 . 처음부터 설계하면 투표와 결제를 같은 <code>user_id</code> 로 추적할 수 있다	달성 40표 스키마
③ 서비스 기획·배포를 경험한다	실데이터를 만들려면 실제로 쓰이는 서비스가 있어야 한다. 기획 · 개인정보 설계 · 배포 · 클로드 테스트까지	달성 W0~W19 · 배포 됨
④ 운영자가 볼 대시보드를 만든다	이탈률 · 결제 현황을 자주 확인해야 하므로 실시간 접근 가능한 화면이 필요하다. 주간 보고를 매번 손으로 만들지 않는다	달성 루커 5쪽

기능 판단이 갈릴 때의 기준

"분석할 데이터가 남는가." 예를 들어 이용자가 여러 학교로 흩어지는 것은 **제품에는 손해지만 분석에는 학교 차원이 생기는 이득**이다. 그래서 흩어지게 두었고, 후보가 모자라면 스코프를 낮추는 대신 다른 친구로 채우고 그 수를 `padded_count` 에 남겼다.

02 · 출발점 — 구 서비스에서 막힌 것

구 Ping 서비스 DB(`final` · `hackle` , 677,085명 · 4천만 행+)를 먼저 분석했다. 리포트 8편에서 구조적 결함 10가지가 나왔고, **결정 문서 16건이 이 분석을 인용**한다.

발견	왜 분석이 막혔나
매출과 핵심 기능이 완전히 분리 결제자 59,192명 중 99.3% 가 투표 불가 학교 소속	투표 × 결제 교차분석이 구조적으로 불가능 . 데이터를 더 얻어도 안 풀린다
탈퇴 기록에 유저 식별자가 없다 70,764건에 <code>reason</code> · <code>created_at</code> 만	탈퇴 사유를 유저 속성과 교차할 수 없다. 이탈 분석의 근거가 사라진다
5,951개 학교 중 딱 10곳에서만 돌아간다 투표 활동 유저 5,090명 (전체의 0.75%)	12개월로 나누면 월 400명 — 월별·이탈구간별 패턴을 볼 표본이 안 된다
신고와 제재가 단절 피신고 253회 유저도 정상 상태	모더레이션 효과 분석 불가
가입자의 63.9%가 1주일 내 이탈 2023-05 코호트의 30일+ 잔존율 3.9%	발견 자체는 유효 — 이 프로젝트의 출발 근거가 됐다

기존 데이터를 "버린" 것이 아니다 — 역할이 달랐다

파이프라인 원천으로 쓰지 않은 이유는 규모가 아니라 데이터가 2024-05에 멈춰 있어서다. 증분 적재는 새로 생기는 행을 잡는 구조인데, 죽은 덤프에 물리면 증분이 영원히 0행이다. DAG가 매일 0행을 가져오는데 맞아서 0인지 고장나서 0인지 구별할 수 없다 — 그건 파이프라인이 아니라 일회성 마이그레이션이다.

그리고 677,085명의 실유저 데이터를 클라우드에 올려 팀원 4명에게 권한을 주는 것은 원래 수집 목적을 벗어난 이용이다. 로컬에서 읽기 전용으로 분석하는 것과 다른 행위다.

03 · 전체 연표 — 28일, 다섯 국면

2026-07-29 ~ 07-31 · 커밋 76개

① 스키마와 웹앱을 한 번에 세웠다

DDL 40표 · RLS 보안 게이트 · 익명 로그인 · 온보딩 · 친구 5명 게이트 · 투표 · 힌트 · **Vercel 배포**까지 사흘. 이어서 게시판 · 친구 추천 · 프로필 · 하트 충전 · 선택형 힌트 · 1회성 답장 · 학사일정 · 운영자 표 제거 · **친구 끊기(W19)** 까지 끝냈다. 같은 기간에 **P4 BigQuery 첫 적재**(789만 행)와 **문서의 위키 재편도** 했다.

2026-08-03 ~ 08-06 · 커밋 41개

② 합성 데이터를 다섯 번 다시 만들었다

v1(1억 4,126만 · 결함 8건) → v2(전면 감사 · 결함 3건) → v3(**확정 직전 중단**) → v4(확정했다가 정밀 EDA에서 결함 1건·경계 5건) → **v5 확정**(1억 2,370만). **2026-08-06에 BigQuery로 전량 적재** — 40개 표 대조 일치. 팀원 4명 접속 개통.

2026-08-11 ~ 08-13 · 커밋 8개

③ 만든 데이터로 실제 분석을 돌렸다

AARR 노트북 — 가입(acquisition) · 활성화(activation) · 리텐션(retention) · 매출(revenue). 이 과정에서 **낡은 DDL 주석**을 발견해 고쳤다(힌트 요금을 반년 전 폐기된 누진제로 설명하고 있었다).

2026-08-18 ~ 08-20 · 커밋 15개

④ 품질 검증과 stg / mart 층을 세우고 DAG로 넘겼다

P5 품질검사 430항목 두 원천 전부 통과 · **P6** stg 뷰 10개(저장 0) + mart 표 8개(495 MiB). 그래인은 **대시보드의 주간 필터가 정했다**. 마트 굽기를 DAG로 넘기고, 품질 검증을 **매일 증분 / 주 1회 전수**로 갈랐다. 지도용 ISO 코드도 이때 stg에 실었다.

2026-08-21 ~ 08-25 · 커밋 4개

⑤ 대시보드를 붙이고 문서를 닫았다

P7 — 루커 스튜디오 5쪽 완성. mart 8표만 연결했다. AARR 노트북 설명 보강 · 합성 데이터 생성 명세서 PDF · 이 문서.

연표에서 읽히는 것 — 순서가 뒤집혀 있다

웹앱이 먼저고 파이프라인이 나중이다. 보통은 반대로 하지만, 이 프로젝트는 **실데이터가 흐르는 구조**가 있어야 파이프라인이 의미를 갖는다고 봤다. 지인 20~50명이 만드는 데이터는 규모가 작지만, **"살아 있는 원천"**은 그것뿐이었다.

04 · 완주표

웹앱 트랙 — W0 ~ W19 (20단계 전부 완료)

단계	내용	남긴 것
W0~W1	익명 인증 스키마 · RLS 보안 게이트	읽기는 필요한 것만, 쓰기는 거의 열지 않는다. 하트·투표 조작은 전부 RPC

W2~W4	앱 뼈대 · 온보딩 · 친구(5명 게이트)	id로 남을 지목할 경로를 만들지 않는다 — 초대 코드로만
W5~W6	투표 · 받은 투표 · 힌트 구매	voter_id 는 유료 정보라 뷰로만 노출
W7	배포 · 클로즈드 테스트	Vercel 자동 배포. 동적 라우트가 안 열려 쿼리스트링으로
W8~W16	학교 정보 — 급식표 · 학사 일정	NEIS 실데이터. 시간표·공지는 남았다
W9	자유게시판 (닉네임 노출)	익명은 안 열었다 — 글쓰기가 붙으면 전제가 바뀐다
W10	친구 추천 (같은 학교)	보안 원칙을 좁은 범위에서 연다. 서버 재확인이 안전장치 전부
W11~W13	프로필 수정 · 계정 삭제 · 하트 충전	W1의 GRANT UPDATE 를 회수했다 — 온보딩 검증을 우회했다
W14~W15	선택형 힌트 · 1회성 답장	누진 요금 폐기 → 5+1 골라 사기
W17~W18	운영자 표 제거 · 이름 정리	42 → 41 → 40표
W19	친구 끊기	지우지 않고 ended_at . 정합성 검사 3종이 남았음이 여기서 드러났다

파이프라인 트랙 — P0 ~ P7

단계	내용	상태	실측
P0	스키마 · DDL	완료	40표 · 342컬럼
P1	합성 데이터 생성	완료	v5 · 20,000명 · 12개월 · 1억 2,370만 행 · 정합성 17종 위반 0
P2	Postgres 적재	완료	순서 정의 40표. 시퀀스 재동기화·위터마크 백필 자동 실행
P3	NEIS 수집	일부	학교·학급·급식·학사일정 완료. DAG 화는 남았다
P4	BigQuery 적재	완료	40표 · 행 수 대조 통과 · 1억 2,373만 행 · 8.74 GiB
P5	품질 검증	완료	430/430 통과, 두 원천 전부. 매일 증분 / 주 1회 전수
P6	stg / mart	완료	stg 뷰 10개(저장 0) · mart 표 8개(495 MiB)
P7	대시보드	완료	루커 스튜디오 5쪽 · mart만 연결

28단계 중 27개 완료, P3의 DAG 화 하나가 부분이다.

05 · 산출물 ① 스키마 — 40표

구 서비스에서 발견한 결함을 표 구조로 담았다. 문서에 정의를 복사하지 않고 DDL + 마이그레이션이 진실이라는 규칙을 세웠다.

구 서비스의 결함	새 스키마에서 담은 방법
탈퇴 기록에 유저 식별자 없음	<code>user_withdrawal</code> 에 유저 식별자를 넣는다
신고와 제재가 단절	신고·제재를 FK로 연결하고 정책을 데이터로 정의
하트 잔액과 원장의 불일치	모든 하트 증감에 <code>balance_after</code> 기록
친구 관계를 지워서 이력이 사라짐	끊어도 행을 지우지 않고 <code>ended_at</code>
계정 삭제로 활동 이력이 증발	삭제도 행을 지우지 않는다
후보가 모자라면 질문이 사라짐	스코프를 낮추지 않고 다른 친구로 채우고 <code>padded_count</code> 에 남긴다

표 수의 이력 — 42 → 41(W17: 운영자 표를 `app_user.is_admin` 으로 접음) → 40(W18: 안 쓰는 표 삭제 · 이름 교정). 표를 지우거나 이름을 바꾸면 옛 마이그레이션이 새 스키마 위에서 깨진다 — 살아 있는 DB에서는 안 보이는 함정이라 `replay_check.py` 가 임시 DB에서 전체를 재생해 대조한다.

06 · 산출물 ② 웹앱 — 개인정보를 받지 않는 MVP

배포: <https://ping-v2-lac.vercel.app> · Next.js 16.2 / React 19.2 / TypeScript. TypeScript 4,807줄 · 화면 패널 10개 · 도메인 모듈 10개.

핵심 제약 — 개인정보를 받지 않는다

받지 않는 것	대신
이메일 · 비밀번호	Supabase 익명 계정 (접속 즉시 자동 생성)
전화번호	친구 추가는 초대 코드로
실명	유저가 정하는 닉네임

받는 것은 닉네임 · 성별 · 소속뿐이다. 성별은 받은 투표의 힌트로 파는 정보라 온보딩 필수 항목이다. ⚠ 이 조합은 소규모 집단에서 개인 특징이 가능하다. 비개인정보라고 단정하지 않고 개인정보처리방침에 수집 항목으로 명시했다.

보안 원칙 — W1에서 정하고 끝까지 지켰다

- 가입도 수정도 RPC 하나뿐이다. `app_user` 에는 INSERT도 UPDATE도 열려 있지 않다. W1에서 `GRANT UPDATE` 로 열어줬다가 W11에서 회수했다 — 그 경로는 온보딩 검증을 전부 우회했다
- id로 남을 지목할 경로를 만들지 않는다. `app_user.id` 는 1부터 이어지는 정수다. 친구 요청 INSERT를 열었더니 코드 없이 전체 가입자를 지목할 수 있었다
- 컬럼을 가려야 하면 뷰를 쓴다. RLS는 행 단위라 컬럼을 못 숨긴다. `voter_id` ("누가 나를 뽑았나")가 하트를 받고 파는 유료 정보이기 때문

기술로 막았지만 남은 구멍 — 소속은 자기신고다

게시판의 학교 경계는 다른 학교 계정으로 14경로를 뚫어봐 전부 막힌 것을 확인했다. 그러나 온보딩에서 아무 학교나 고를 수 있고 익명 계정은 무제한이라 "그 학교 사람인가"는 보증되지 않는다. 불특정 다수에게 열 때 먼저 볼 항목으로 남겼다.

07 · 산출물 ③ 합성 데이터 — 아홉 판본의 기록

규모: 20,000명 · 50개교 · 834학급 · 240문항 · 12개월 · 1억 2,370만 행 · 40표 전부. 비율(투표율 · 결제 전환율)은 구 서비스 실측에서 가져왔다.

판본	행수	무엇이 드러났나
1개월 v1~v3	500명 샘플	첫 검증 · 하트 문제의 성격이 뒤집힘 · 페르소나 도입 · 우측 절단 발견
v1	1억 4,126만	성장 곡선 첫 구현. 결함 8건
v2	1억 8,826만	전면 감사 · 결함 3건
v3	1억 8,918만	확정 직전 <code>user_session</code> 결함이 나와 중단
v4	1억 8,922만	목표 10개 전부 달성해 확정했다가, 정밀 EDA에서 결함 1건·경계 5건
v5	1억 2,370만	확정. 요구 7가지 충족 · 목표 지표 전부 통과

v4 → v5에서 행이 35% 줄어든 것은 없어야 했던 것이 빠진 것이다

v4에는 하루에 406.8시간 투표한 유저-일이 있었다. 하루는 24시간이다.

개수 상한(12회)을 걸어 봤더니 해비유저가 통째로 사라져 행이 반토막 나고 지목 쓸림이 39%로 주저앉았다 — 결함을 고친 게 아니라 분포를 죽인 것이다. 그래서 시간 예산으로 바꿨다. 사람이 하루에 낼 수 있는 시간만 제약하고, 그 안에서 몇 번을 하든 내버려 둔다.

설계의 핵심 — 사람을 먼저 만들고, 그 사람에게서 로그가 나오게 한다

각 행동을 독립적으로 무작위 생성하면 상관관계가 없는 데이터가 나온다. 그러면 "결제하는 사람은 어떤 사람인가"에 답이 안 나온다 — 답이 애초에 안 심겨 있기 때문이다.

페르소나 6유형 × 트레잇 10종이 그 장치다. 그리고 일부러 흐린다 — 혼합 25% · 트레잇 교차 8% · 무배정 4% · 트레잇마다 따로 거는 로그정규 편차. 코드 주석의 실패 조건이 이렇다:

"결과 데이터에 6덩어리가 또렷하게 남으면 실패다. 분석자가 클러스터링해서 우리가 넣은 6개를 그대로 되찾는 데이터는 발견할 것이 없다."

정답지 `data/personas.json` (5MB)은 DB에도 CSV에도 넣지 않는다. 현재 용도는 "맞았나" 채점이 아니라 "너무 또렷하지 않은가" 검증이다.

만드는 법 전체(설정값 · 노이즈 · 게이트)는 `synthetic-data-build-spec.pdf` 17장, 왜 그 값인지는 `synthetic-data-decision-history.pdf` 30문답에 있다.

이 데이터로 할 수 없는 것 7가지

v4에서는 15가지였고 **v5가 9종을 열었다**. 남은 7가지는 변동계수까지 재서 문서에 박았다 — 질문별 인기 · A/B(표본 노이즈) · 슬롯 위치 편향(0.0039) · 문항 위치별 품질(0.0002) · 성별 상호작용 · 게시판 카테고리 (0.0046) · 앱 버전·기기 · 광고 네트워크.

08 · 산출물 ④ 파이프라인 — 세 층과 DAG 셋



층	무엇	규칙
raw	40표 · 1억 2,373만 행 8.74 GiB	원본 보존. 삭제는행을 지우지 않고 <code>_deleted_at</code> 으로 표시. 실유저와 합성이 같은 표에 들어가고 <code>_source</code> 로 키를 나눈다 — 둘 다 id가 1부터라 실제로 겹친다
stg	뷰 10개 · 저장 0	원천 구분 · 타입 · 시간대를 정리한다. 대리키 <code>user_key ('local-123')</code> 로 잇게 만들어 <code>USING(id)</code> 자체를 불가능하게 했다 — 합정을 구조적으로 없앴 것
mart	표 8개 · 495 MiB	그레인을 정한 것은 대시보드의 주간 필터다. 아래 규칙 넷

주간 규칙 넷 — 마트 설계를 결정한 것

"주 단위로 좁혀 전주와 비교한다"가 붙는 순간 월 그레인 · 사전 계산된 비율 · 미리 센 `distinct`가 전부 못 쓰게 된다.

규칙	여기면
① 모든 표에 날짜 컬럼(<code>metric_date</code>)	루커의 '비교 기간(전주)'이 아예 안 켜진다
② 비율을 굵지 않는다. 분자·분모를 심는다	주간 비율이 "일별 비율의 평균"이 된다
③ <code>distinct</code> 를 사전 집계하지 않는다	월 표에서 주간 MAU를 못 낸다. DAU를 7일 더해도 WAU가 아니다
④ 퍼널·분포는 사건일 또는 코호트일로 자른다	기간을 좁힐 때 같은 대상이 두 주에 중복된다

그래서 `mart_daily`에는 활성 유저 수가 일부러 없다 — 사람 수는 날짜 축으로 더해지지 않기 때문이다. 사람 세는 일은 전부 `mart_user_activity`가 한다.

Airflow DAG 셋

DAG	하는 일
<code>ping_raw_load</code>	Supabase → BigQuery raw 증분 적재 + 행 수 대조
<code>ping_quality_full</code>	품질 검증 — 매일 증분 / 주 1회 전수로 갈랐다
<code>ping_mart_build</code>	mart 굽기. 앞 DAG를 손대지 않고 뒤에 붙였다

담당자가 매번 SQL을 손으로 돌리면 누락된다. DAG가 적재 → 검사 → 마트 생성 순서를 강제해 의존성을 보장한다. Cloud Composer는 월 40만원대라 로컬 Docker로 띄웠다.

09 · 산출물 ⑤ 대시보드 — 루커 스튜디오 5쪽

P7 완료. `mart` 데이터셋만 연결했다. 설계대로 5쪽이 그대로 붙었다.

쪽	내용	쓰는 마트
1	KPI · 시계열	<code>mart_user</code> · <code>mart_user_activity</code> · <code>mart_daily</code> · <code>mart_funnel_step</code>
2	유저 · 코호트	<code>mart_user</code> (유저 1명 그레인, 축 = 가입일)
3	퍼널 · 분포	<code>mart_funnel_step</code> · <code>mart_distribution</code>
4	하트 유입 · 소비처	<code>mart_heart_flow</code>
5	신고 · 적체	<code>mart_report</code> · <code>mart_backlog_weekly</code>

"설계대로 붙었다"가 이 단계의 성과다

P7은 코드가 필요 없는 단계다. 파이썬은 P6에서 끝난다. 그래서 이 단계에서 아무 일도 안 일어났다는 것은 앞에서 한 준비가 맞았다는 뜻이다.

구체적으로 세 가지가 미리 되어 있었다 — ① 주간 규칙 넷을 지켜 그레인을 잡아 뒀고, ② 여덟 표가 같은 차원 이름(`source` · `sido` · `school_type` · `grade` · `gender`)을 써서 보고서 수준 필터가 한 번에 걸리고, ③ 지도용 ISO 코드를 `stg`에 미리 실어 마트 여덟 개가 자동으로 물려받았다.

불일 때 지킨 것 셋

항목	이유
<code>raw</code> 를 연결하지 않는다	루커는 화면을 새로 그릴 때마다 쿼리를 날린다. 필터를 한 번 바꾸면 한 번 더 날아간다. <code>raw</code> 를 붙이면 그때마다 1억 2천만 행을 읽는다 — 월 1 TiB 무료 한도가 이렇게 사라진다
원천 필터를 고정한다	실유저는 23명이라 합성과 섞이면 비율이 무의미해진다. 보고서 수준 필터로 <code>source</code> 를 고정
지도는 <code>sido_iso</code> 로	한글 시도명으로는 매칭이 안 붙는다 — 강원특별자치도 처럼 개편된 이름은 확실히 실패한다. 못 알아본 값은 NULL로 남긴다 — 틀린 코드를 넣으면 엉뚱한 지역이 칠해지고 아무도 눈치채지 못한다

⚠ 대시보드가 뜻을 갖는 범위

지도는 합성 데이터에서만 뜻이 있다. 실유저 23명은 전원 서울이다. 그리고 이 지역 값은 학교를 거쳐 붙는 자기신고다 — 실유저 사본에서는 지도를 빼는 편이 낫다.

퍼널은 도형이 아니라 막대 묶음으로 그렸다. `user_journey` 는 단조 감소하지 않는다 — 단계가 독립 플래그라 "투표 세션 완료 13,873 → 1단계 힌트 15,210"처럼 다시 늘어난다. 힌트는 받은 투표에서 사는 것이라 투표 세션과 인과가 없기 때문이다.

10 · 산출물 ⑥ 문서 체계 — 89개 결정 노드

문서를 세 층으로 갈랐다. 어느 층을 건드리는지 늘 의식한다.

층	무엇	고치는 주체	규모
1	raw/ 원본 — 회의록 · 구 서비스 분석 · 외부 스펙	사람만. 에이전트는 읽기만	분석 리포트 8편 + PDF 2
2	docs/ 위키 — 원본과 코드에서 뽑은 것	에이전트가 소유	결정 89 · 운영 9 · 표 40 · PDF 18
3	CLAUDE.md 규약 — 그 일을 어떻게 하는지	사람 + 에이전트	—

raw/ 는 고치지 않는다. 회의에서 정한 것이 나중에 바뀌어도 회의록은 그대로 둔다. 바뀐 것은 새 결정으로 적는다 — 그래야 "언제 왜 마음이 바뀌었나"가 남는다. 실제로 4건이 superseded 로 뒤집혔다.

세 가지 연산 — ingest · query · lint

query는 전문을 훑지 않는다. grep으로 파일명만 받고, 이름을 보고 골라 그 노드만 읽는다. 실측 토큰 — 골라 읽기 1,263 · 한 파일 시절 grep 1,928 · 매칭 노드 전부 읽기 6,907.

⚠ 이득을 만드는 것은 구조가 아니라 규율이다. 매칭된 노드를 전부 읽으면 쪼개기 전보다 나쁘다. 그래서 되돌리는 길도 함께 만들었다 — wiki_merge.py 가 지금 있는 노드 전부를 한 파일로 합친다 (git revert 로 하면 그 뒤에 쓴 결정이 함께 사라진다).

lint는 문서의 주장을 살아 있는 DB·파일시스템과 대조한다. 아홉 가지를 본다 — 숫자 · 사라진 이름 · 끊어진 링크 · 없는 스크립트 · 낡은 색인 · 미반영 회의록 · 없는 질문 번호 · 결정 없이 쌓인 코드 · 색인에서 빠진 결정 노드.

뒤의 셋은 실제로 겪은 것을 잡으려고 나중에 붙였다. 합성 데이터 작업에서 결정 노드를 아홉 번 연속으로 건너뛰었고, 근거가 커밋 메시지와 설정 주석에만 남았기 때문이다.

이 위키가 다른 위키와 다른 점

원본이 불변인 것은 같은데, 여기서는 코드도 진실이고 코드는 매 커밋 바뀐다. 그래서 위키가 조용히 낡는다 — 오류가 안 나므로 더 위험하다. lint 연산이 그것 때문에 있다.

11 · 검증 체계 — 층층이 쌓은 검사 다섯

이 프로젝트에서 검사가 코드만큼 늘었다. 전부 실제로 당하고 나서 붙인 것이다.

검사	항목	무엇을 막나
db/rls/verify.py	201	침투 · 동작 시험. 배포 전 반드시 통과. 양방향으로 본다 — 전부 막아도 침투 시험은 통과하므로 정상 동작 시험이 함께 있어야 뜻이 있다

qa/checks/integrity.sql	17	정합성. 위반 0이어야 한다
qa/quality_check.py	430	적재 데이터 품질. 두 원천 전부 통과. 한 번에 11.7 GiB 스캔
db/doc_lint.py	9	문서의 주장 vs 실제
생성 게이트	5	정합성 · 342컬럼 커버리지 · 40표 커버리지 · 증분 실증 · 분포 리포트

⚠ 시험이 전부 통과해도 앱이 죽을 수 있다 — safeupdate

2026-07-30에 실제로 그랬다. 브라우저는 PostgREST를 거쳐 `authenticator` 역할로 접속하고, 그 역할에는 `safeupdate` 가 걸려 있어 **WHERE 없는 DELETE/UPDATE를 임시 테이블에서도 막는다.**

그런데 `verify.py` 는 `postgres` 로 붙어 `SET LOCAL ROLE` 만 한다. `preload`는 **세션이 열릴 때** 적용되므로 역할만 바뀌어서는 안 걸린다. 그래서 `verify.py` 가 소스를 직접 검사하도록 고쳤다.

교훈 — 역할을 바꾸는 것과 그 역할로 접속하는 것은 다르다.

검사도 낡는다 — W19에서 3종이 한꺼번에 터졌다

친구 끊기를 열자 평소에 안 드러나던 검사 오류 3개가 나왔다. **셋 다 데이터가 아니라 검사가 틀린 경우였다.** 게이트 위반 검사는 "지금 친구<5인데 해금"으로 물었는데 설계는 한 번 찍으면 유지한다. 힌트 구매 검사는 **하트 0인 광고 무료 힌트**를 위반으로 잡았다.

반대로 **12번은 일부러 안 고쳤다.** "투표 당시 친구였는가"를 묻는 검사라 `ended_at IS NULL` 을 붙이면 친구를 끊는 순간 옛 투표가 전부 위반이 된다.

교훈 — 기능을 열 때 그 기능이 건드리는 검사를 함께 읽는다. 검사는 쓰인 시점의 세계를 알고 있고, 그 세계는 기능이 늘 때마다 바뀐다.

12 · 실패 기록 — 조용히 틀린 것들

이 프로젝트에서 실제로 배운 것은 성공한 설계가 아니라 **여기 있는 것들**이다. 공통점이 하나 있다 — 대부분 오류로 나타나지 않는다.

A. 규모에서만 나오는 것

사건	무엇이 일어났나
파티션 한도 초과 2026-08-06	BigQuery는 파티션 테이블 하나에 하루 4,000회의 파티션 수정 만 받는다. 세는 단위가 작업이 아니라 작업 × 파티션 이라는 것이 함정이다. 저장 순서대로 꺼내니 20만 행 한 덩어리가 30일치에 흩어져 131건 만에 찼고 <code>vote_candidate</code> 가 2,620만 행에서 403. 같은 적재에서 <code>vote_item</code> 은 작업이 68건뿐이라 통과했다 — 표 크기가 아니라 작업 수 × 파티션 폭이 경계선 이다. 고침 — <code>ORDER BY</code> 파티션 컬럼. 부수 효과로 속도가 1.8배(초당 12,732 → 23,215 행). "끝으로 갈수록 느려지던" 것은 성능 특성이 아니라 한도가 차오르는 신호였다.
"적재는 무료"가 아니었다 2026-08-20	하루 쿼리 하드캡을 채웠다. 잡 이력을 뒤져 보니 44.59 GiB 중 44.45 GiB가 MERGE-UPDATE 였다. 루커는 1,190건에 5.31 GiB로 오히려 썼다. 배치 로드는 무료지만 그 뒤의 MERGE와 소프트 삭제 UPDATE는 과금 된다. 둘 다 <code>_source</code> PK로 거르는데 파티션 컬럼이 <code>updated_at</code> 이라 가지치기가 안 걸린다 — 증분으로 한 줄만 바뀌어도 표를 통째로 훑는다. 전날 계산한 월 269 GiB는 이 항목을 0으로 놓고 센 것이라 틀렸다(실제 약 600).
행수 상한이 아무것도 안 막았다	<code>max_total_rows</code> (1억)를 코드가 읽지 않았다. 생성기는 1억 4천만 행을 경고 없이 만들었다. 게다가 값 자체가 틀렸다 — 8,500만 추정은 유저 수와 기간이 같이 움직이는 두 점으로 맞춘 규칙이라 기간만 12배로 늘리자 깨졌다 . 실측은 유저당 4,251행이 아니라 7,063행 . 고침 — 행수 상한을 버리고 BigQuery 커스텀 할당량으로. 지키지 않는 상한은 상한이 아니다.

B. 증분이 조용히 틀리는 네 경로

경로	증상
컬럼 추가 후 정지	<code>ALTER TABLE ADD COLUMN</code> 은 트리거를 발동시키지 않아 <code>updated_at</code> 이 그대로다. 증분은 "바뀐 거 없네" 하고 0행을 가져온다 → <code>vote_item.padded_count</code> 가 합성 803,187행 전부 NULL 이었다. 오류 0.
과거 시각 백필	위터마크는 "그 시각 0/후"만 보는데 값이 뒤로 가면 영영 안 잡힌다 .
지운 행의 유형	증분은 "바뀐 행"은 가져와도 "없어진 행"은 못 본다 . 탈퇴한 계정과 그 활동이 오늘 것으로 샘해진다. <code>_deleted_at</code> 으로 표시하되 쿼리마다 IS NULL 을 넣어야 한다 — 2026-07-30에 실제로 물렸다.
<code>_source</code> 없는 조인	두 원천의 id가 실제로 겹친다 — <code>app_user</code> 16개, <code>vote_item</code> 26개. 고침 — <code>stg</code> 가 대리키를 쓰게 만들어 <code>USING(id)</code> 자체를 불가능하게 했다.

위터마크를 지우는 것이 핵심이다. 합성을 지울 때 행만 지우고 `_load_state` 를 남기면 다음 적재가 "이미 최신"으로 판단해 **아무것도 안 올리고 조용히 끝난다**. 에러도 안 난다.

C. 대량 적재가 파티션을 무의미하게 만든 것

`updated_at` 의 기본값은 `now()` 다. COPY로 786만 행을 부으면 **786만 행이 전부 "적재한 순간"**이 된다. 3개월치가 하루에 뭉치고, BigQuery 파티션 가지치기가 통째로 무력해져 쿼리마다 786만 행을 다 읽는다.

함정 하나 더 — 되돌릴 때 트리거를 꺾다 켜야 한다. 켜둔 채 UPDATE 하면 트리거가 방금 넣은 값을 다시 `now()` 로 덮어써서 **아무 일도 하지 않은 것처럼 끝난다**. 나중에 백필 UPDATE가 3시간을 먹어서, 아예 **생성기가 처음부터 올바른 시각을 심도록** 바꿨다.

D. 문서와 코드가 어긋난 것

사건	무엇이 틀렸나
DDL 주석이 반년 남아 있었다 2026-08-13	<code>30_question_vote.sql</code> 의 주석을 근거로 힌트 요금을 200→300→500→1000 누진이라고 말했다. W14가 반년 전에 없앤 요금제다. <code>apply.py</code> 가 DDL + 마이그레이션 을 전부 돌리므로 <code>db/ddl/</code> 은 현재 스키마가 아니라 출발 상태 다 — 진실은 파일 하나가 아니라 순서에 있다. 고침 — 남은 블록에 경고를 그 자리에 남겼다. 주석만 고친다 — SQL 문장은 재생성의 출발점이라 현행으로 "고치면" 마이그레이션이 깨진다.
옛 마이그레이션이 새 스키마에서 깨졌다 2026-07-31 · 2건	009는 DDL이 더 이상 만들지 않는 표를 찾다가, 010은 이미 있는 트리거를 다시 만들다가 실패했다. 둘 다 Supabase에서는 멀쩡했다 — 거기엔 옛 표가 실제로 있었기 때문이다. 고침 — <code>replay_check.py</code> 가 임시 DB에서 전체를 재생해 대조한다.
결정을 아홉 번 안 적었다	합성 데이터 작업에서 결정 노드를 연속으로 건너뛰었고 근거가 커밋 메시지와 설정 주석에만 남았다. 고침 — lint에 "마지막 결정 이후 코드 커밋 8개 초과" 검사를 붙였다.

E. 설정에 값이 있어도 코드가 안 읽으면 없는 것 — 아홉 번

2026-08-03 ~ 05 사흘 동안 아홉 번 뺏았다. **전부 오류가 안 나고 데이터를 다 만든다**. 분석해야 드러난다.

죽어 있던 설정	증상
<code>schools.tiers</code> · <code>adoption</code>	학교 50곳이 전부 첫날 열렸다
Q23 시간대	0~23시가 전부 4.1~4.2% (완전 균등)
<code>weekend_multiplier</code> · <code>hour_weights</code>	요일 효과가 아예 없었다 (변동계수 0.024)
<code>growth</code> · <code>reactivation_rate</code> · <code>top10_share</code> · <code>behavior</code>	성장 곡선 무시 · 복귀 유저 0명 · 목표치 미적용 · <code>seasonality</code> 가 밖으로 밀려남

구조적으로 막은 방법 셋 — ① `load_config()` 의 절 복사 방식을 뒤집었다 ② 값을 **코드 옆 상수**로 옮기고 설정으로는 켜고 끄기만 한다 ③ **죽은 설정은 지운다** — 다시는 안 읽히는 값이 남지 않게.

설정 파일은 선언이지 보증이 아니다. 값이 있다는 것과 그 값이 결과에 도달했다는 것은 다른 얘기이고, 둘의 차이는 오류로 나타나지 않는다.

13 · 숫자로 본 프로젝트

구분	값	비고
기간	28일	작업일 15일 · 커밋 143개
완료 단계	27 / 28	웹앱 20 + 파이프라인 8 (P3 DAG화만 부분)
스키마	40표 · 342컬럼	42 → 41 → 40으로 줄여 왔다
Python	10,319줄	생성기 2,998 · RLS 검증 1,689 외
SQL	6,665줄	DDL · 마이그레이션 · RLS · stg 10 · mart 8
TypeScript	4,807줄	화면 패널 10 · 도메인 모듈 10
BigQuery raw	1억 2,373만 행	8.74 GiB — 무료 한도(10 GiB)의 87%
stg / mart	뷰 10 / 표 8	저장 0 / 495 MiB
대시보드	5쪽	루커 스튜디오 · mart만 연결
검사 항목	662	RLS 201 + 품질 430 + 정합성 17 + lint 9 + 게이트 5
결정 노드	89	뒤집힌 것 4 · 운영 노드 9 · 표 페이지 40
PDF 리포트	18	EDA 8 · 프로젝트 4 · 설명서 6
합성 판본	9	1개월 샘플 3 + 12개월 6 (v5 확정)

14 · 관통하는 교훈 다섯

① 데이터 엔지니어링의 버그는 터지지 않는다. 조용히 틀린다

웹앱 버그는 화면이 안 뜨니까 바로 안다. 파이프라인 버그는 초록불인 채로 틀린 숫자를 내놓는다. 12장의 사건 대부분이 그랬다. 그래서 "돌았는가"가 아니라 "맞는가"를 확인하는 검사를 662개 만들었다.

② 파이프라인은 배관이지 정수기가 아니다

연결고리가 없는 데이터는 적재해도 안 생긴다. 구 서비스 데이터를 BigQuery에 올리고 stg·mart를 세워도 답할 수 있는 질문은 그대로다. 그래서 스키마부터 다시 설계했다.

③ 아래층에서 미리 한 준비가 위층의 작업량을 결정한다

P7에서 아무 일도 안 일어난 것이 증거다. 주간 규칙 넷으로 그래인을 잡고, 여덟 표에 같은 차원 이름을 쓰고, ISO 코드를 stg에 미리 실어 뒀기 때문에 대시보드는 그냥 붙었다. 반대로 그 준비가 없었다면 마트를 다시 구웠어야 했다.

④ 이득을 만드는 것은 구조가 아니라 규율이다

위키로 쪼갠 것이 토큰을 34% 아꼈지만, 매칭된 노드를 전부 읽으면 쪼개기 전보다 나쁘다(6,907 vs 1,928). 같은 이유로 검사도, 설정 파일도, 문서도 **지키지 않으면 없는 것이다**. 그래서 되돌리는 길 (`wiki_merge.py`)과 판단 기준을 함께 남겼다.

⑤ 한계를 먼저 재서 적으면 나머지가 신뢰를 얻는다

합성 데이터로 할 수 없는 것 7가지를 변동계수까지 재서 문서에 박았다. 빈 표가 왜 비었는지도 `EMPTY_REASON` 에 적고, 안 적으면 화면이 "사유 미기재"로 드러낸다. 데이터를 만드는 일보다 그 데이터를 어디까지 믿을지 정하는 일이 더 오래 걸렸다.

15 · 남은 일 — 정직하게

순위	항목	내용
1	P3 NEIS DAG 화	학교·학급·급식·학사일정은 받았지만 수집이 아직 손으로 돈다. 28단계 중 유일하게 부분인 항목
2	앱과 생성기의 어긋남 3건	투표 보상 · 일일 적립 상한 · 이름 공개 시 <code>reveal_status</code> 미갱신. 앱을 생성기에 맞추기로 했고 다음 판본에서 한 번에 고친다
3	W8 학교 정보 절반	급식·학사일정은 붙였고 시간표·공지 화면이 없다. 표는 이미 있다
4	처리 경로 없는 신고	<code>sanction</code> 은 신고가 PENDING 으로 고인다. 표만 있고 화면이 없다. <code>block_record</code> · <code>question_request</code> 도 같다
5	클로즈드 테스트 확대	실유저 23명. 초대할 미를 이유는 없어졌다 — 증분이 처음부터 흐르므로 소급 적재가 필요 없다

열기 전에 먼저 있어야 할 것

하기로 했지만 순서가 없는 기능이 셋 있다 — 익명 메시지 · 채팅방 · (이미 연) 게시판. 셋 다 자유 텍스트가 사람에게 직접 간다. 익명 게시판을 뺐던 이유가 그대로 적용되고, 1:1 메시지는 공개 게시판보다 위험하다 — 사적인 메시지는 받은 사람이 말하기 전까지 아무도 모른다. 열기 전에 차단·신고 화면이 먼저 있어야 한다.

그리고 학교 소속은 자기신고다. 기술로는 막혀 있지만(14경로 확인) 불특정 다수에게 열 때 이것이 첫 번째로 볼 항목이다.

16 · 이 프로젝트가 증명한 것

목표는 실서비스 운영이 아니라 데이터 파이프라인과 분석이었다. 웹앱은 실데이터를 만드는 수단이고, 클로즈드 테스트는 그 데이터를 얻는 방법이었다. 그 기준으로 보면 결과는 이렇다.

증명한 것	근거
전 구간을 혼자 세울 수 있다	스키마 40표 → 웹앱 배포 → 실데이터 → 합성 1억 2,370만 행 → raw/stg/mart → DAG 3개 → 대시보드 5쪽. 28단계 중 27개 완료
규모에서 깨지는 것을 만나고 고쳤다	파티션 한도 · 쿼리 하드캡 · 증분 워터마크 4경로 · 파티션 가지치기 무력화. 지인 20~50명 데이터로는 하나도 못 만난다
데이터를 의심할 줄 안다	합성 v4를 확정했다가 정밀 EDA로 되돌렸다(하루 406.8시간). 자기가 만든 데이터의 한계 7가지를 변동계수까지 재서 적었다. 페르소나가 너무 또렷하지 않은지를 정답지로 검증했다
운영을 자동화했다	DAG가 적재 → 검사 → 마트 순서를 강제한다. 품질 검증을 매일 증분 / 주 1회 전수로 갈라 비용과 신뢰성을 함께 잡았다
맥락을 재현 가능하게 남겼다	결정 89개가 이유·대안·영향과 함께 노드로 남아 있고, 4건은 뒤집힌 이력까지 남는다. <code>doc_lint</code> 9종이 문서의 주장을 살아 있는 DB와 대조한다

마지막으로 — 이 프로젝트의 진짜 산출물

대시보드 5쪽이 아니다. **12장의 실패 기록**이다.

파이프라인을 만드는 법은 문서를 보면 된다. 만들어 본 사람만 아는 것은 **어디서 조용히 틀리는가**이고, 그것을 **날짜와 숫자와 함께** 남긴 것이 이 저장소가 가진 다른 저장소와 다른 점이다.

ping-v2 · 프로젝트 완주 기록 · 2026-08-25

더 깊이 — 입문 `PROJECT-REPORT.pdf` (12장) · 합성 데이터 만드는 법 `synthetic-data-build-spec.pdf` (17장) · 왜 그 값인가 `synthetic-data-decision-history.pdf` (30문답) · 검수 `EDA-final-12m-v5.pdf` (23쪽) · 파이프라인 운영 `docs/ops/ops-p5-p7.md`